

Control de Robot Seguidor de Línea con Procesamiento Externo

Material

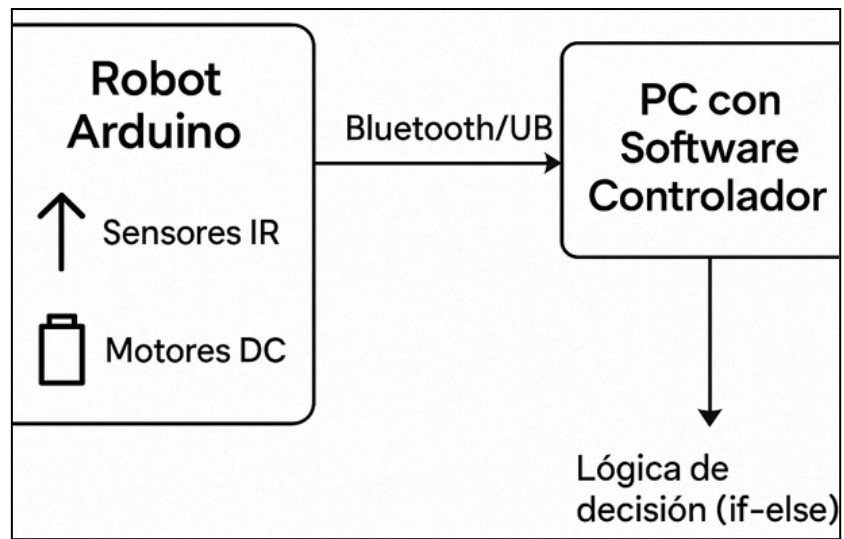
- Plataforma Arduino UNO o equivalente.
- Módulo Bluetooth (HC-05/HC-06) o cable USB-serial.
- Módulo de sensores infrarrojos (3 o más).
- Motores y chasis (kit de robot básico).
- Computadora con software de control (Python + pyserial, C#, etc.).
- Cinta negra o superficie con línea trazada para prueba.

Desarrollo técnico

División del sistema

Función	Responsable	Justificación
Captura de sensores IR	Hardware	La lectura de sensores infrarrojos requiere recolección rápida y frecuente de señales analógicas o digitales. Es más eficiente que el microcontrolador lo haga directamente, ya que está físicamente conectado a los sensores y puede capturar datos en tiempo real con mínima latencia.
Control de motores	Hardware	La activación de motores mediante señales PWM o digitales debe ser precisa y oportuna. Delegar esta función al hardware asegura que las órdenes se ejecuten de forma inmediata sin depender de la comunicación con la PC.
Lógica de seguimiento (decisión)	Software (PC)	Centralizar la lógica de navegación (decisiones de movimiento) en la PC permite mayor flexibilidad para cambiar o probar algoritmos de control, facilita la depuración y permite aprovechar más poder de cómputo para implementar lógicas avanzadas.
Comunicación y protocolo	Ambos	La transmisión de datos debe ser coordinada. El hardware se encarga de enviar datos de sensores y ejecutar comandos; el software interpreta datos y envía decisiones. Ambos extremos deben compartir el protocolo para asegurar la correcta interpretación de mensajes.

Diagrama del Sistema



Asignacion

Función	Componente asignado	Detalles
Captura de sensores IR	Arduino UNO	Pines digitales para lectura de sensores IR, utilizando <code>digitalRead()</code> o <code>analogRead()</code> si se usan sensores analógicos.
Control de motores (PWM/dirección)	Arduino UNO	Controlador de motor (L298N o puente H), señales PWM desde pines ~3, ~5, ~6, etc.
Comunicación con PC	Módulo USB-Serial (Arduino integrado)	Uso de <code>Serial.print()</code> para enviar datos de sensores y <code>Serial.read()</code> para recibir decisiones desde la PC.
Decisión de movimiento (lógica)	Aplicación Python en PC	Interpreta datos enviados por Arduino, ejecuta algoritmo de línea negra (e.g. "sensor central detecta línea → avanzar") y envía comandos.
Interfaz de usuario (visualizar estado)	Aplicación Python en PC	Visualiza en consola o GUI simple (Tkinter/Qt) el estado de sensores y acciones tomadas.

Refinamiento Arduino UNO:

- Código C++ usando setup() y loop()
- Lectura de sensores IR cada 10 ms
- Envío del estado de los sensores vía Serial.print()
- Recepción de órdenes: 'F' (forward), 'L' (left), 'R' (right), 'S' (stop)
- Accionamiento de motores vía PWM (usar funciones como moverAdelante(), girarIzquierda(), etc)

Software en PC (Python):

- Uso de pyserial para comunicarse con Arduino
- Algoritmo de decisión: si solo sensor central = línea → adelante, si sensor izquierdo → girar, etc.
- GUI simple con logs o botones de prueba
- Posibilidad de registrar sesiones o comportamiento en archivos .txt o .csv

Flujo de comunicación

1. Arduino lee valores de sensores IR (ej: izquierda = 0, centro = 1, derecha = 0).
2. Envía cadena "010" por Bluetooth o puerto serie.
3. PC recibe el valor, decide el movimiento:
 - "010" → avanzar
 - "100" → girar izquierda
 - "001" → girar derecha
 - "000" o "111" → detener o giro de corrección
4. PC envía comando al Arduino: "F" (Forward), "L" (Left), "R" (Right), "S" (Stop).
5. Arduino interpreta y ejecuta el movimiento.

Código a implementar en Arduino

```
char command;
void setup() {
  Serial.begin(9600);
  // setup motores y sensores
}
void loop() {
  if (Serial.available()) {
    command = Serial.read();
    switch(command) {
      case 'F': avanzar(); break;
      case 'L': girarIzq(); break;
      case 'R': girarDer(); break;
```

```

        case 'S': detener(); break;
    }
}
}

```

Código a Implementar en Python

```

import serial
arduino = serial.Serial('COM3', 9600)
def decidir_direccion(sensor):
    if sensor == "010":
        return "F"
    elif sensor == "100":
        return "L"
    elif sensor == "001":
        return "R"
    else:
        return "S"
while True:
    sensor_data = arduino.readline().decode().strip()
    comando = decidir_direccion(sensor_data)
    arduino.write(comando.encode())

```

Métodos de verificación sugeridos

Simulación del algoritmo en software (sin Arduino):

- Crear una función en Python que reciba combinaciones binarias de sensores (101, 010, etc.) y devuelva acciones.
- Prueba con múltiples entradas simuladas para verificar que el algoritmo responde correctamente.

Pruebas con Arduino desconectado (modo loopback):

- Simular envío de cadenas "010", "100" desde la PC y verificar cómo respondería el motor virtualmente.
- Así puedes validar que el algoritmo y la comunicación funcionan antes de cargar al microcontrolador.

Simulación de entorno físico:

- Usar entornos como Tinkercad Circuits o Proteus para montar sensores IR virtuales y verificar señales de entrada/salida.
- Simulación de motores mediante LEDs o actuadores virtuales.

Pruebas parciales del sistema real:

- Validar solo sensores IR conectados a Arduino.
- Luego probar solo motores.
- Finalmente, integrar comunicación serial y verificar paso a paso.

Criterios funcionales de verificación:

- ¿Se leen correctamente los sensores?
- ¿Se interpreta la señal correctamente en PC?
- ¿Se reciben y ejecutan comandos en el Arduino?
- ¿El robot se comporta de forma coherente ante cada entrada simulada o real?